TravelLink Frontend Architecture Plan

> **Goal:** Build a mobile-first, responsive React frontend that efficiently consumes all backend endpoints with a premium, modern UI.

---

1. Backend API Inventory (Complete Scan)

Controllers & Endpoints

Module	Route Prefix	Auth	Endpoints
Auth	`api/auth`	Public	`POST register`, `POST login`, `POST refresh`
User	`api/user`	JWT	`GET profile`, `POST avatar`
Friend	`api/friend`	JWT	`POST send`, `POST respond/{id}?action=`, `GET pending`, `GET list`, `DELETE unfriend/{id}`
Group	`api/group`	JWT	`POST create`, `GET my`, `GET {id}`, `POST {id}/add-member`, `DELETE {id}/leave`, `DELETE {id}/remove-member`, `POST {id}/cover`
Expense	`api/expense`	JWT	`POST add`, `GET {id}`, `DELETE {id}`, `GET group/{gid}`, `GET group/{gid}/balances`, `GET group/{gid}/analytics`, `PUT split/{sid}/pay`, `GET user/{uid}/summary`
Itinerary	`api/itinerary`	JWT	`POST request`, `GET group/{gid}/status`, `DELETE`

Module	Route Prefix	Auth	Endpoints
			`request/{rid}`, `POST {rid}/suggest`, `DELETE suggestion/{sid}`, `POST suggestion/{sid}/vote`, `PATCH suggestion/{sid}/review`, `GET {rid}/suggestions`, `POST {rid}/generate`, `POST request/auto-generate`, `GET {rid}/result`, `PATCH item/{iid}/complete`

SignalR Hub

Hub	Path	Auth	Methods
LocationHub	`/hubs/location`	JWT (query param)	`JoinGroupSession(groupId)`, `JoinPrivateSession(targetUserId)`, `SendLocation(dto)`, `LeaveSession()`
			Client Events: `LocationUpdate`, `UserJoined`, `UserLeft`, `SessionLeft`, `Error`

ServiceResult Wrapper

All endpoints return: `{ success: boolean, message?: string, data?: T }`

> [!IMPORTANT] Some controllers (User, Group, Friend) don't consistently wrap responses in `ServiceResult`. The frontend API layer must handle **both** raw and wrapped responses gracefully.

2. Recommended Tech Stack

Layer	Technology	Rationale
Framework	React 19 + Vite	Already in use, excellent DX and HMR
Language	TypeScript (strict)	Type safety matching backend DTOs
Styling	Tailwind CSS v4	Already installed, mobile-first utility classes
Routing	React Router v7	Already installed, nested layouts support
HTTP Client	Axios	Already installed, interceptors for ServiceResult unwrapping
Real-time	@microsoft/signalr	Already installed, for LocationHub
Toasts	Sonner	Already installed, for ServiceResult error/success messages
Icons	Lucide React	Already installed, consistent icon system
Maps	Leaflet + react-leaflet	Already have leaflet installed, for location tracking
State	React Context + useReducer	Sufficient for this app size; no Redux needed
Charts	Recharts (to install)	For expense analytics (pie charts, bar graphs)

3. Folder Structure (Mobile-First Architecture)

```
src/
├── components/              # Shared reusable UI components
│   ├── ui/                  # Primitives: Button, Input, Modal, Card, Avatar,
Badge
│   ├── layout/              # AppShell, MobileNav, Sidebar, TopBar
│   └── ProtectedRoute.tsx
│
├── context/                 # React Contexts for global state
│   ├── AuthContext.tsx       # User session, tokens, login/logout
│   └── LocationContext.tsx   # SignalR connection, live location state
│
├── features/                # Feature-based modules (each self-contained)
│   ├── auth/
│   │   ├── LoginPage.tsx
│   │   └── RegisterPage.tsx
│   │
```

```
│   ├── dashboard/
│   │   └── DashboardPage.tsx     # Overview: groups, pending requests, quick
actions
│   │
│   ├── friends/
│   │   ├── FriendsPage.tsx       # Friend list + pending requests (tabs)
│   │   └── components/
│   │       ├── FriendCard.tsx
│   │       ├── FriendRequestCard.tsx
│   │       └── SendRequestModal.tsx
│   │
│   ├── groups/
│   │   ├── GroupsPage.tsx        # List of my groups
│   │   ├── GroupDetailPage.tsx   # Single group view (members, itinerary,
expenses)
│   │   └── components/
│   │       ├── GroupCard.tsx
│   │       ├── CreateGroupModal.tsx
│   │       ├── MemberList.tsx
│   │       └── AddMemberModal.tsx
│   │
│   ├── expenses/
│   │   ├── GroupExpensesPage.tsx  # Expenses for a specific group
│   │   ├── UserSummaryPage.tsx   # User-to-user expense summary
│   │   └── components/
│   │       ├── ExpenseCard.tsx
│   │       ├── CreateExpenseModal.tsx
│   │       ├── BalanceList.tsx
│   │       ├── AnalyticsCharts.tsx    # Pie/bar charts for category breakdown
│   │       └── SplitDetails.tsx
│   │
│   ├── itinerary/
│   │   ├── ItineraryPage.tsx         # Main itinerary view for a group
│   │   ├── AutoGeneratePage.tsx      # Auto-generate form (wizard-style)
│   │   └── components/
│   │       ├── ItineraryTimeline.tsx  # Day-by-day timeline view
│   │       ├── ItineraryItemCard.tsx
│   │       ├── SuggestionCard.tsx
│   │       ├── SuggestionList.tsx
│   │       ├── BudgetSummary.tsx
│   │       └── DroppedSuggestions.tsx
│   │
│   ├── location/
│   │   ├── LocationPage.tsx          # Full-screen map view
│   │   └── components/
│   │       ├── LiveMap.tsx           # Leaflet map with markers
│   │       ├── UserMarker.tsx
│   │       └── SessionControls.tsx   # Join/leave session buttons
│   │
│   └── profile/
│       └── ProfilePage.tsx           # Avatar upload, user info
│
├── services/                # API service layer (1 file per backend controller)
│   ├── api.ts               # Axios instance + interceptors
│   ├── authService.ts
│   ├── userService.ts
│   ├── friendService.ts
│   ├── groupService.ts
│   ├── expenseService.ts
│   ├── itineraryService.ts
│   └── locationService.ts   # SignalR connection manager
│
├── types/                   # TypeScript interfaces (mirrors backend DTOs)
│   ├── index.ts             # Re-exports everything
```

```
│   ├── auth.ts
│   ├── user.ts
│   ├── friend.ts
│   ├── group.ts
│   ├── expense.ts
│   ├── itinerary.ts
│   └── location.ts
│
├── hooks/                    # Custom React hooks
│   ├── useApi.ts             # Generic fetch hook with loading/error states
│   └── useDebounce.ts
│
├── App.tsx                   # Root component with routing
├── main.tsx                  # Entry point
└── index.css                 # Tailwind v4 imports + custom design tokens
```

---

4. TypeScript Interfaces (All Backend DTOs)

`types/auth.ts`

```
export interface LoginDto {
    email: string;
    password: string;
}

export interface RegisterDto {
    name: string;
    email: string;
    password: string;
}

export interface AuthResponse {
    token: string;
    refreshToken: string;
    name: string;
    email: string;
    userId: string;
}

export interface RefreshTokenRequest {
    token: string;
    refreshToken: string;
}
```

`types/user.ts`

```
export interface UserProfile {
    id: string;
    name: string;
    email: string;
    imageUrl?: string;
}
```

`types/friend.ts`

```
export interface FriendDto {
    userId: string;
    name: string;
```

```
    email: string;
    imageUrl?: string;
}

export interface FriendRequestDto {
    id: string;
    senderId: string;
    senderName: string;
    senderEmail: string;
    status: string;
    sentAt: string;
}

export interface SendFriendRequestDto {
    receiverId: string;
}
```

types/group.ts

```
export interface GroupDto {
    id: string;
    name: string;
    description?: string;
    coverImageUrl?: string;
    createdAt: string;
    createdByUserId: string;
    createdByName: string;
    members: GroupMemberDto[];
}

export interface GroupMemberDto {
    userId: string;
    userName: string;
    email: string;
    role: string;       // "Admin" | "Member"
    joinedAt: string;
}

export interface CreateGroupDto {
    name: string;
    description?: string;
    coverImageUrl?: string;
    memberIds: string[];
}
```

types/expense.ts

```
export interface CreateExpenseDto {
    title: string;
    description?: string;
    amount: number;
    category: string;
    date: string;
    groupId?: string;
    splitType: string;          // "Equal" | "Custom"
    participantIds: string[];
    splits: ExpenseSplitInputDto[];
}

export interface ExpenseSplitInputDto {
    userId: string;
    amountOwed: number;
```

```
}

export interface ExpenseDto {
    id: string;
    title: string;
    description?: string;
    amount: number;
    category?: string;
    date: string;
    createdAt: string;
    paidByUserId: string;
    paidByUser: string;
    paidByImageUrl?: string;
    groupId?: string;
    splits: ExpenseSplitDto[];
    warning?: string;
}

export interface ExpenseSplitDto {
    id: string;
    userId: string;
    name: string;
    userImageUrl?: string;
    amountOwed: number;
    isPaid: boolean;
    paidAt?: string;
}

export interface BalanceDto {
    fromUserId: string;
    fromUserName: string;
    fromUserImageUrl?: string;
    toUserId: string;
    toUserName: string;
    toUserImageUrl?: string;
    amount: number;
}

export interface GroupAnalyticsDto {
    totalGroupSpend: number;
    totalSettled: number;
    totalUnsettled: number;
    totalExpenses: number;
    categoryBreakdown: CategoryBreakdownDto[];
    memberContributions: MemberContributionDto[];
    balances: BalanceDto[];
}

export interface CategoryBreakdownDto {
    category: string;
    totalAmount: number;
    expenseCount: number;
    percentage: number;
}

export interface MemberContributionDto {
    userId: string;
    userName: string;
    imageUrl?: string;
    totalPaid: number;
    totalOwed: number;
    netBalance: number;
}
```

```
export interface UserToUserDto {
    userId: string;
    name: string;
    imageUrl?: string;
    netBalance: number;
    totalYouOwe: number;
    commonGroups: string[];
    transactions: UserTransactionDto[];
}

export interface UserTransactionDto {
    expenseId: string;
    title: string;
    amount: number;
    category: string;
    date: string;
    youPaid: boolean;
    yourShare: number;
    isSettled: boolean;
    groupName?: string;
}
```

types/itinerary.ts

```
export interface CreateItineraryRequestDto {
    groupId: string;
    destination: string;
    startDate: string;
    endDate: string;
    totalBudget: number;
    dailyHotelCostPerRoom?: number;
    numberOfRooms?: number;
    vehicleType?: string;
    vehicleCount?: number;
    isRental?: boolean;
    dailyRentalCostPerVehicle?: number;
    dailyFuelCostPerVehicle?: number;
}

export interface AutoGenerateItineraryDto extends CreateItineraryRequestDto {
    note?: string;
}

export interface GenerateItineraryDto {
    note?: string;
}

export interface AddSuggestionDto {
    name: string;
    type: string;       // "place" | etc.
    notes?: string;
}

export interface ReviewSuggestionDto {
    adminApproved?: boolean;
}

export interface GroupItineraryStatusDto {
    requestId: string;
    destination: string;
    startDate: string;
    endDate: string;
    totalBudget: number;
```

```
    status: string;             // "Open" | "Generating" | "Generated"
    totalSuggestions: number;
    approvedSuggestions: number;
}

export interface SuggestionDto {
    id: string;
    name: string;
    type: string;
    notes?: string;
    adminApproved?: boolean;
    voteCount: number;
    hasCurrentUserVoted: boolean;
    suggestedByName: string;
    suggestedByImageUrl?: string;
    suggestedByUserId: string;
    createdAt: string;
}

export interface ItineraryResultDto {
    id: string;
    requestId: string;
    destination: string;
    startDate: string;
    endDate: string;
    totalDays: number;
    groupSize: number;
    generatedAt: string;
    totalBudget: number;
    hotelTotalCost: number;
    dailyHotelCost: number;
    numberOfNights: number;
    replacedExisting: boolean;
    vehicleTotalCost: number;
    estimatedActivityCost: number;
    estimatedTotalSpend: number;
    budgetRemaining: number;
    estimatedCostPerPerson: number;
    vehicleType?: string;
    vehicleCount?: number;
    dailyVehicleCost: number;
    days: ItineraryDayDto[];
    droppedSuggestions: DroppedSuggestionDto[];
}

export interface ItineraryDayDto {
    id: string;
    dayNumber: number;
    date: string;
    title: string;
    weatherNote?: string;
    items: ItineraryItemDto[];
}

export interface ItineraryItemDto {
    id: string;
    orderIndex: number;
    type: string;
    placeName: string;
    description: string;
    durationMinutes: number;
    notes: string;
    category: string;
    estimatedCostPerPerson?: number;
```

```
    bookingSearchQuery?: string;
    bookingType?: string;
    isCompleted: boolean;
}

export interface DroppedSuggestionDto {
    name: string;
    reason: string;
}
```

`types/location.ts`

```
export interface LocationUpdateDto {
    userId: string;
    userName: string;
    imageUrl?: string;
    lat: number;
    lng: number;
    accuracy: number;
    speed?: number;
    timestamp: string;
}
```

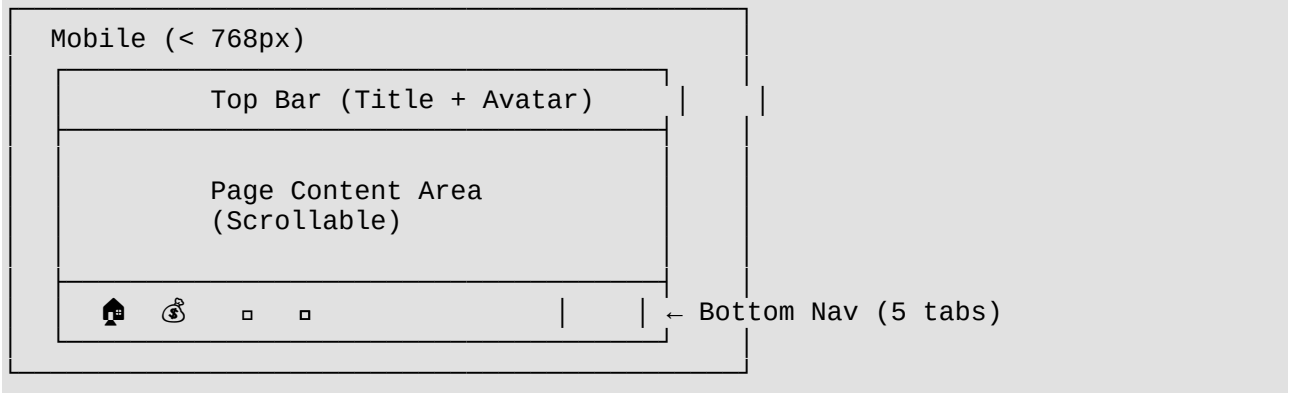
`types/index.ts` (Re-export barrel)

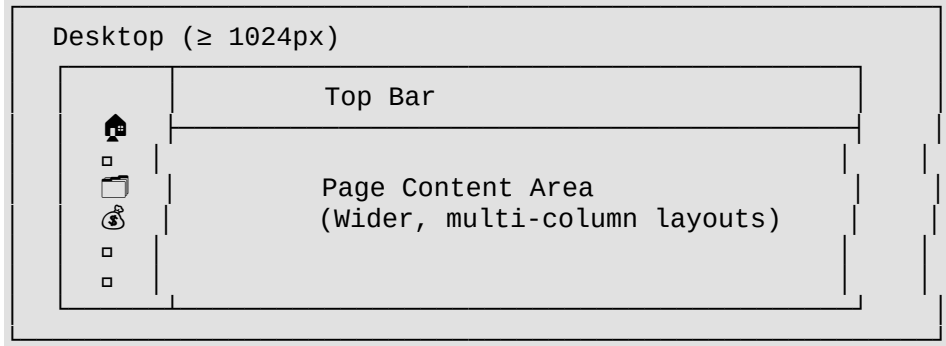
```
export * from './auth';
export * from './user';
export * from './friend';
export * from './group';
export * from './expense';
export * from './itinerary';
export * from './location';

// Generic ServiceResult wrapper
export interface ServiceResult<T = any> {
    success: boolean;
    message?: string;
    data?: T;
}
```

---

5. Mobile-First Layout Strategy

Navigation Pattern: Bottom Tab Bar (Mobile) + Sidebar (Desktop)





Bottom Tab Targets (Mobile)

Tab	Icon	Route	Page
Home	`Home`	`/dashboard`	Dashboard overview
Friends	`Users`	`/friends`	Friends + requests
Groups	`FolderPlus`	`/groups`	My groups list
Expenses	`Wallet`	`/expenses`	Quick expense overview
Map	`MapPin`	`/location`	Live location

---

6. Routing Plan

```
<Routes>
  {/* Public */}
  <Route path="/login" element={<LoginPage />} />
  <Route path="/register" element={<RegisterPage />} />

  {/* Protected - Wrapped in AppShell (layout with nav) */}
  <Route element={<ProtectedRoute />}>
    <Route element={<AppShell />}>
      <Route path="/dashboard" element={<DashboardPage />} />

      {/* Friends */}
      <Route path="/friends" element={<FriendsPage />} />

      {/* Groups */}
      <Route path="/groups" element={<GroupsPage />} />
      <Route path="/groups/:groupId" element={<GroupDetailPage />} />

      {/* Expenses (within group context) */}
      <Route path="/groups/:groupId/expenses" element={<GroupExpensesPage />} />
      <Route path="/expenses/user/:userId" element={<UserSummaryPage />} />

      {/* Itinerary (within group context) */}
      <Route path="/groups/:groupId/itinerary" element={<ItineraryPage />} />
      <Route path="/groups/:groupId/itinerary/auto-generate"
element={<AutoGeneratePage />} />

      {/* Location */}
      <Route path="/location" element={<LocationPage />} />
```

```
      {/* Profile */}
      <Route path="/profile" element={<ProfilePage />} />
    </Route>
  </Route>

  {/* Fallback */}
  <Route path="*" element={<Navigate to="/dashboard" />} />
</Routes>
```

> [!TIP] The `<AppShell />` component wraps protected pages and renders the **bottom nav bar on mobile** and **sidebar on desktop**. It uses `<Outlet />` internally to render child routes.

---

7. API Service Layer Design

`services/api.ts` — Axios Instance with Global Interceptors

Key Logic:

1. **Request Interceptor:** Automatically attaches the JWT `Authorization: Bearer <token>` header.
2. **Response Interceptor:** If a 401 is returned, attempt a silent token refresh using the stored `refreshToken`. If refresh fails, fire a global `auth_logout` event (which the `AuthContext` listens for).
3. **ServiceResult Unwrapping:** Can optionally unwrap `ServiceResult` at the interceptor level, or leave it to individual services.

Per-Module Services

Each service file will mirror its backend controller:

Service File	Methods
`authService.ts`	`login()`, `register()`, `refresh()`
`userService.ts`	`getProfile()`, `uploadAvatar()`
`friendService.ts`	`sendRequest()`, `respond()`, `getPending()`, `getFriends()`, `unfriend()`
`groupService.ts`	`create()`, `getMyGroups()`, `getById()`, `addMember()`, `leave()`, `removeMember()`, `uploadCover()`
`expenseService.ts`	`create()`, `getById()`, `delete()`, `getGroupExpenses()`, `getGroupBalances()`, `getGroupAnalytics()`, `markSplitPaid()`, `getUserToUserSummary()`

Service File	Methods
`itineraryService.ts`	`createRequest()`, `getGroupStatus()`, `deleteRequest()`, `addSuggestion()`, `deleteSuggestion()`, `toggleVote()`, `reviewSuggestion()`, `getSuggestions()`, `generate()`, `autoGenerate()`, `getResult()`, `toggleItemComplete()`
`locationService.ts`	`connect()`, `joinGroupSession()`, `joinPrivateSession()`, `sendLocation()`, `leaveSession()`, `disconnect()`, `onLocationUpdate()`, `onUserJoined()`, `onUserLeft()`

---

8. Implementation Phases (Recommended Order)

Phase 1: Foundation (Do First)

- ☐ Scaffold Vite + React + TypeScript project
- ☐ Install all dependencies
- ☐ Configure Tailwind v4 + PostCSS correctly
- ☐ Set up `src/types/` with all interfaces
- ☐ Set up `services/api.ts` with Axios interceptors (including token refresh)
- ☐ Set up `AuthContext` with login/register/logout/refresh
- ☐ Create `ProtectedRoute`

Phase 2: Auth UI

- ☐ Build `LoginPage` (premium split-screen design)
- ☐ Build `RegisterPage`
- ☐ Test full login → redirect → dashboard flow

Phase 3: App Shell & Dashboard

- ☐ Build `AppShell` with responsive bottom nav / sidebar
- ☐ Build `DashboardPage` (overview cards: groups count, pending friend requests, upcoming trips)
- ☐ Build shared UI components: `Button`, `Input`, `Modal`, `Card`, `Avatar`, `Badge`

Phase 4: Friends Module

- ☐ `friendService.ts`
- ☐ `FriendsPage` with tabs: My Friends | Pending Requests
- ☐ Send friend request (search by ID or email)
- ☐ Accept/reject requests
- ☐ Unfriend

Phase 5: Groups Module

- ☐ `groupService.ts`
- ☐ `GroupsPage` (card grid of groups)
- ☐ `GroupDetailPage` (tabbed: Members | Expenses | Itinerary | Location)
- ☐ Create group modal, add/remove members
- ☐ Cover image upload

Phase 6: Expenses Module

- ☐ `expenseService.ts`
- ☐ `GroupExpensesPage` (expense list + create modal)
- ☐ Balance cards (who owes whom)
- ☐ Analytics page with charts (install `recharts`)
- ☐ Mark split as paid
- ☐ `UserSummaryPage` (user-to-user transaction history)

Phase 7: Itinerary Module

- ☐ `itineraryService.ts`
- ☐ Collaborative flow: create request → suggestions → vote → review → generate
- ☐ Auto-generate wizard (multi-step form for budget, transport, dates)
- ☐ Generated itinerary timeline view (day-by-day with items)
- ☐ Toggle item completion
- ☐ Budget summary cards
- ☐ Dropped suggestions list

Phase 8: Location Module

- ☐ `locationService.ts` (SignalR connection manager)
- ☐ `LocationContext` for real-time state

- ☐ Full-screen Leaflet map
- ☐ Group session and private session controls
- ☐ Live user markers with avatars

Phase 9: Profile & Polish

- ☐ Profile page with avatar upload
- ☐ Dark mode toggle
- ☐ Loading skeletons across all pages
- ☐ Empty state illustrations
- ☐ PWA manifest for mobile install

---

9. Key Design Decisions

Why Feature-Based Folders over Type-Based?

- Each feature (friends, groups, expenses) is self-contained with its own components
- Easy to find all related code in one place
- Scales better as the app grows
- Prevents a bloated `components/` folder with 50+ files

Why Context + useReducer over Redux/Zustand?

- Only 2 truly global states needed: Auth + Location
- Everything else is page-level fetched data (loaded on mount)
- Keeps bundle size small and architecture simple
- Can upgrade to Zustand later if needed

Why Bottom Tab Nav for Mobile?

- Travel apps are primarily mobile experiences
- Thumb-reachable navigation (bottom of screen)
- Matches patterns users know (Google Maps, Splitwise, TripAdvisor)
- 5 tabs = perfect fit for your 5 main modules

> [!NOTE] The `GroupDetailPage` acts as a **hub page** — from a single group, users can access that group's expenses, itinerary, location sharing, and member management via internal tabs. This eliminates deep nesting in the mobile nav.